

WEEK 02 MODULE

Programming Practice

2. Server Log Search (Binary Search)

que.1 What is Searching?

- Searching is the process of finding whether a particular element exists in a collection of data or not.

Types of Searching :-

- i. Linear Search
- ii. Binary Search

- **Binary Search** is a searching technique that finds an element by repeatedly dividing a sorted array into two halves.

Key Points :-

- Works on sorted arrays only.
- How it works:
 1. Find the middle element.
 2. If middle = target → Found.
 3. If target is smaller → Search left half.
 4. If target is larger → Search right half.
 5. Repeat until found or array becomes empty.

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists

1. Problem Analysis

The cloud company maintains server log IDs in sorted order. Engineers need to verify whether a particular log ID exists within the stored records.

Since the log IDs are already arranged in ascending order, an efficient searching technique can be used instead of checking each element sequentially. The objective is to locate the target log ID quickly while minimizing the number of comparisons.

2. Algorithm Selection Analysis

Since the server log IDs are already stored in sorted order, Binary Search is the most suitable searching technique for this problem.

Unlike Linear Search, which examines elements one by one, Binary Search repeatedly divides the search space into two halves. This significantly reduces the number of comparisons required to locate the target log ID.

Reasons for selecting Binary Search:

- The input data is already sorted.
- Search operations become faster for large datasets.
- Half of the remaining elements are eliminated after each comparison.
- Efficient for systems that frequently perform search operations on ordered records.

Therefore, Binary Search provides a more optimized solution for locating server log IDs compared to sequential searching techniques.

3. Core Idea / Logic

Binary Search works by comparing the target log ID with the middle element of the sorted array.

- If the middle element matches the target, the search is successful.
- If the target is smaller than the middle element, the search continues in the left half of the array.
- If the target is greater than the middle element, the search continues in the right half of the array.
- This process repeats until the target is found or the search space becomes empty.

By eliminating half of the remaining elements during each iteration, the search becomes much faster than checking every element individually.

4. Execution Flow

Input

- Read the number of server log IDs (n).
- Store all sorted log IDs in an array.
- Read the target log ID to be searched.

Processing

- Initialize:
 - $left = 0$
 - $right = n - 1$
- Calculate the middle index.
- Compare the middle element with the target log ID.
- Update the search boundaries based on the comparison result.
- Repeat until the target is found or the search space is exhausted.

Output

If the target log ID exists: Log Found

Otherwise: Log Not Found

Step-by-Step Program Execution

Using the sample input:

```
7
1001 1005 1010 1015 1020 1030 1040
1015
```

Step 1

Read the number of log IDs.

User enters 7. ($n=7$)

```
int n = sc.nextInt();
 $1 \leq n \leq 10^5$ 
```

means:

$1 \leq n \leq 100000$

In simple words:

- Minimum value of n can be **1**
- Maximum value of n can be **100000**

Step 2

Create an array of size n .

Array of size 7 is created.

```
int arr[] = new int[n];
```

Step 3

Accept all sorted log IDs from the user and store them in the array.

```
1001 1005 1010 1015 1020 1030 1040
```

Array becomes:

Index :	0	1	2	3	4	5	6
value :	1001	1005	1010	1015	1020	1030	1040

Step 4

Read the target log ID.

```
int target = sc.nextInt();
```

```
target = 1015
```

Step 5

```
Initialize:      left = 0;  
                right = n - 1;
```

Step 6

Calculate the middle index.

```
mid = (left + right) / 2;
```

Step 7

Compare the middle element with the target log ID.

Step 8

If the middle element equals the target:

- Mark it as found.
- Stop the search.

Step 9

If the target is smaller than the middle element:

- Move the search to the left half.

Step 10

If the target is greater than the middle element:

- Move the search to the right half.

Step 11

Repeat the process until the target is found or no search space remains.

Step 12

Display the appropriate result message.

5. Key Observations

- The array **must be sorted** before Binary Search can be applied.
- The algorithm does not examine every element individually.
- During each comparison, approximately half of the remaining search space is eliminated.
- The search range continuously decreases using the left and right boundaries.
- Best-case scenario occurs when the target log ID is located at the middle position during the first comparison.
- Worst-case scenario occurs when multiple divisions are required before locating the target log ID or determining that it does not exist.
- Binary Search is significantly more efficient than Linear Search for large sorted datasets.
- The algorithm continues until either the target is found or the search range becomes empty (left > right).
- Performance depends on the sorted nature of the data; an unsorted array would produce incorrect results.
- Binary Search is well-suited for applications involving frequent searches on ordered records, such as server logs, databases, and indexing systems.

6. Program Structure Analysis

i. Variable n

int n;

Purpose:

- Stores the total number of server log IDs entered by the user.
- Determines the size of the array.
- Defines the search range for Binary Search.

$$1 \leq n \leq 10^5$$

means:

$$1 \leq n \leq 100000$$

In simple words:

- Minimum value of n can be **1**
- Maximum value of n can be **100000**

ii. Array arr[]

int arr[] = new int[n];

Purpose:

- Stores all server log IDs.
- Since Binary Search requires sorted data, the array contains log IDs in ascending order.
- Allows indexed access to elements during the search process.

iii. Variable target

int target;

Purpose:

- Stores the log ID that needs to be searched.
- Acts as the key value for comparison during Binary Search.

iv. Variable left

int left = 0;

Purpose:

- Represents the starting index of the current search range.
- Initially points to the first element of the array.
- Moves right when the target is greater than the middle element.

v. Variable right

int right = n - 1;

Purpose:

- Represents the ending index of the current search range.
- Initially points to the last element of the array.
- Moves left when the target is smaller than the middle element.

vi. Variable mid

int mid;

Purpose:

- Stores the middle index of the current search range.
- Used to divide the array into two halves.
- Helps determine which half should be searched next.

vii. Boolean Variable found

boolean found = false;

Purpose:

- Tracks whether the target log ID has been found.
- Initially assumes the log ID is absent.
- Changes to true when a matching log ID is located.

viii. Input Loop

```
for(int i = 0; i < n; i++)
{
    arr[i] = sc.nextInt();
}
```

Purpose:

- Accepts all server log IDs from the user.
- Stores them in the array.
- Ensures the complete dataset is available before searching begins.

ix. Binary Search Loop

```
while(left <= right)
```

Purpose:

- Continues searching as long as a valid search range exists.
- Terminates when the target is found or when the search range becomes empty.

x. Middle Index Calculation

```
mid = (left + right) / 2;
```

Purpose:

- Finds the middle position of the current search range.
- Divides the array into two halves for comparison.

xi. Equality Check

```
if(arr[mid] == target)
```

Purpose:

- Checks whether the middle element matches the target log ID.
- If true, the search is successful.

xii. Left Half Selection

```
right = mid - 1;
```

Purpose:

- Executed when the target is smaller than the middle element.
- Discards the right half of the array.
- Restricts the search to the left portion.

xiii. Right Half Selection

left = mid + 1;

Purpose:

- Executed when the target is greater than the middle element.
- Discards the left half of the array.
- Restricts the search to the right portion.

xiv. Break Statement

break;

Purpose:

- Stops the search immediately after finding the target log ID.
- Prevents unnecessary iterations.

xv. Output Decision Structure

if(found)

Purpose:

- Determines the final search result.
- Displays the appropriate output message based on whether the target log ID was found.

7. Program

```
package week2_module;
import java.util.Scanner;
public class ServerLogSearchBinarySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input number of log IDs
        System.out.print("Enter the number of server log IDs: ");
        int n = sc.nextInt();
        if(n < 1 || n > 100000){
            System.out.println("Error: N must be between 1 and 100000.");
            sc.close();
            return;
        }

        // Create array
        int arr[] = new int[n];
```



```

// Input sorted log IDs
System.out.print("Enter the " + n + " server log IDs in sorted order : ");
for(int i = 0; i < n; i++){
    arr[i] = sc.nextInt();
}

// Input target log ID
System.out.print("Enter the suspicious of transaction ID: ");
int target = sc.nextInt();
// Initialize variables for Binary Search
int left = 0;
int right = n - 1;
boolean found = false;
// Binary Search
while(left <= right){
    int mid = (left + right) / 2;
    if(arr[mid] == target){
        found = true;
        break;
    }
    else if(target < arr[mid]){
        right = mid - 1;
    }
    else{
        left = mid + 1;
    }
}
// Display result
if(found){
    System.out.println("Log Found");
}
else{
    System.out.println("Log Not Found");
}
sc.close();
}
}

```

8. Program's output

```

Enter the number of server log IDs: 7
Enter the 7 server log IDs in sorted order : 1001
1005
1010
1015
1020
1030
1040
Enter the suspicious of transaction ID: 1015
Log Found

```

9.. Screenshots

- program

```
ServerLogSearchBinarySearch.java X
1 package week2_module;
2 import java.util.Scanner;
3 public class ServerLogSearchBinarySearch {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7         System.out.print("Enter the number of server log IDs: "); // Input number of log IDs
8         int n = sc.nextInt();
9         int arr[] = new int[n]; // Create array
10        System.out.print("Enter the " + n + " server log IDs in sorted order : "); // Input sorted log IDs
11        for(int i = 0; i < n; i++)
12        {
13            arr[i] = sc.nextInt();
14        }
15        System.out.print("Enter the suspicious of transaction ID: "); // Input target log ID
16        int target = sc.nextInt();
17
18        // Initialize variables for Binary Search
19        int left = 0;
20        int right = n - 1;
21        boolean found = false;
22
23        // Binary Search
24        while(left <= right){
25            int mid = (left + right) / 2;
26            if(arr[mid] == target){
27                found = true;
28                break;
29            }
30            else if(target < arr[mid]){
31                right = mid - 1;
32            }
33            else{
34                left = mid + 1;
35            }
36        }
37        // Display result
38        if(found)
39        {
40            System.out.println("Log Found");
41        }
42        else
43        {
44            System.out.println("Log Not Found");
45        }
46        sc.close();
47    }
48 }
49 }
```

- program's output

```
Problems @ Javadoc Declaration Console X
<terminated> ServerLogSearchBinarySearch [Java Application] /
Enter the number of server log IDs: 7
Enter the 7 server log IDs in sorted order : 1001
1005
1010
1015
1020
1030
1040
Enter the suspicious of transaction ID: 1015
Log Found
```